

Figure 11: Spring-mass system

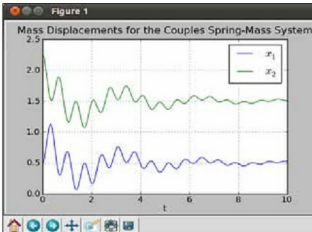


Figure 12: Plot of the spring-mass system

Slot 2 a number of times, determined by a positive integer in Slot 3. Unless a dependency has been established, all parameters and variables in the expression are treated as constants when taking the derivative. Similarly, you can also calculate higher order differentials with Maxima.

Ordinary differential equations

Maxima can also be used to solve ODEs. We'll dive straight into some examples to understand how to solve ODEs with Maxima. Consider the following differential equations:

$$\begin{aligned} dx/dt &= e^{-x} + x \\ d^2x/dt^2 &= -4x \end{aligned}$$

Consider Figure 3. Let's rewrite our example ordinary differential equations using the noun form *diff*, which uses a single quote. Then use *ode2*, and call the general solution *gsoln*.

The function *ode2* solves an ordinary differential equation (ODE) of the first or second order. This takes three arguments: an ODE given by *eqn*, the dependent variable *dvar*, and the independent variable *ivar*. When successful, it returns either an explicit or implicit solution for the dependent variable. *%c* is used to represent the integration constant in the case of first-order equations, and *%k1* and *%k2* the constants for second-order equations.

We can also find the solution at pre-defined points using *ic1* and call this particular solution, *psoln*. Consider the following non-linear first order differential equation:

$$(x^2 y) dy/dx = xy^3 + x^3 - 1$$

Let's first define the equation, and then solve it with *ode2*. Further, let us find the particular solution at points *x=1* and *y=1* using *ic1*.

We can also solve ODEs with NumPy and SciPy using the *FuncDesigner* and *OpenOpt* packages. However, both these packages make use of automatic differentiation to solve ODEs. Hence, Maxima was chosen over these packages. ODEs can also be solved using the *scipy.integrate.odeint* package. We will later use this package for mathematical modelling.

Curve plotting with Matplotlib

It's said that a picture is worth a thousand words, and there's no denying the fact that it's much more convenient to make sense of a scientific experiment by looking at the plots as compared to looking just at the raw data. In this article, we'll be focusing on Matplotlib, which is a Python package for 2D plotting that produces production-quality graphs. Matlab is customisable and extensible, and is integrated with LaTeX markup, which is really useful when writing scientific papers. Let us make a simple plot with the help of Matplotlib:

```

1
2 #Simple Plot with Matplotlib
3
4 #! /usr/bin/python
5
6 import matplotlib.pyplot as plt
7
8 x = range(10)
9
10 plt.plot(x, [xi**3 for xi in x])
11
12 plt.show()
```

Let us take another example using the *arange* function; *arange(x,y,z)* is a part of NumPy, and it generates a sequence of elements with *x* to *y* with spacing *z*.

```

1
2 #Simple Plot with Matplotlib
3
4 #! /usr/bin/python
5
6 import matplotlib.pyplot as plt
7 import numpy as np
```